Course: AI Assisted Coding

Assignment-10.4

Name: V. Sathwik reddy

BATCH-14

HTNO:2303A510D7

Task 1: AI-Assisted Syntax and Code Quality Review

Scenario

You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized.

Task Description

You are given a Python script containing:

• Syntax errors

• Indentation issues

• Incorrect variable names

• Faulty function calls

Use an AI tool (GitHub Copilot / Cursor AI) to:

• Identify all syntactic and structural errors

• Correct them systematically

• Generate an explanation of each fix made

Expected Outcome

• Fully corrected and executable Python code

• AI-generated explanation describing:

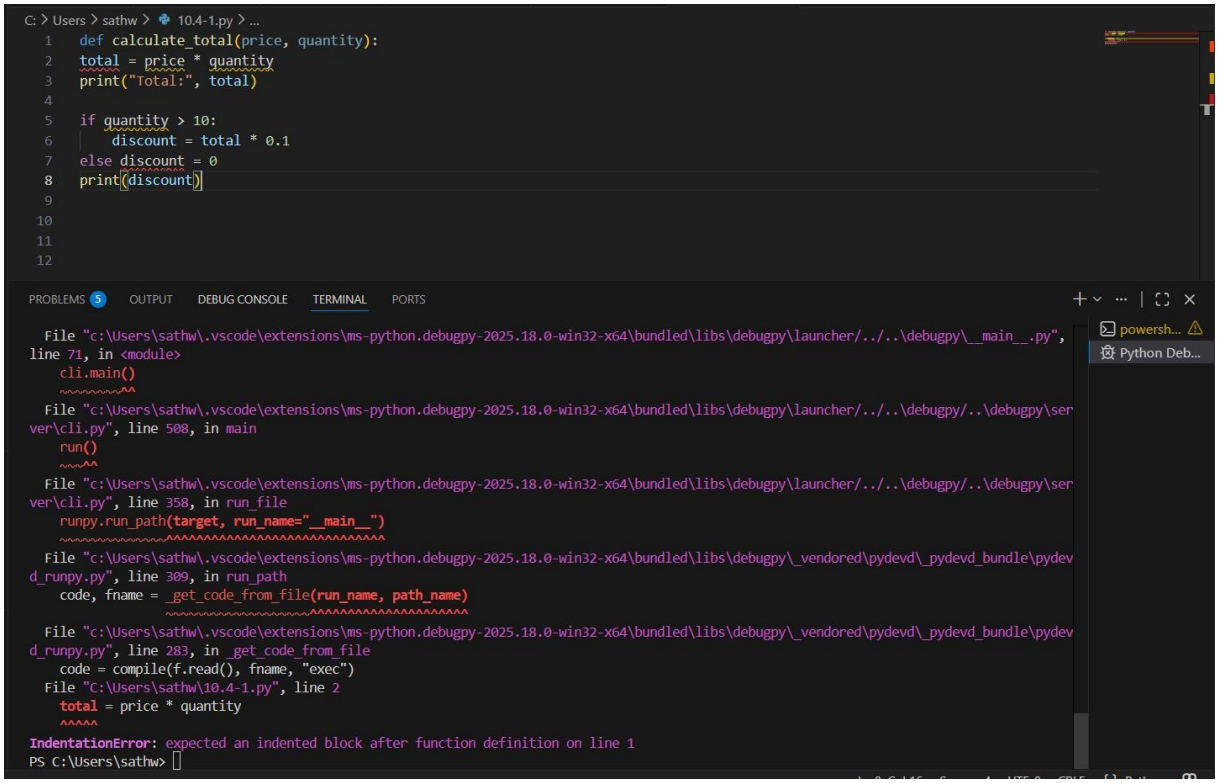
o Syntax fixes

o Naming corrections

o Structural improvements

• Clean, readable version of the script

Task 1 – Wrong Code (Syntax & Structural Errors)



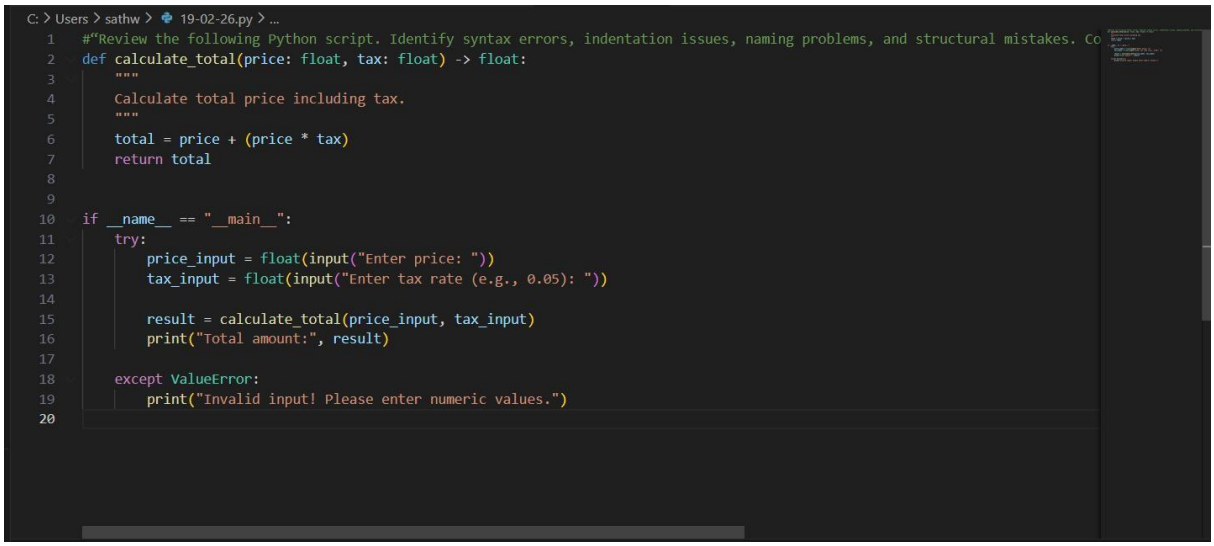
Errors Present

- Missing comma in parameters
- Missing colon
- No indentation
- Python 2 print statement
- Function name mismatch
- Missing comma in function call
- No return statement

Prompt Used

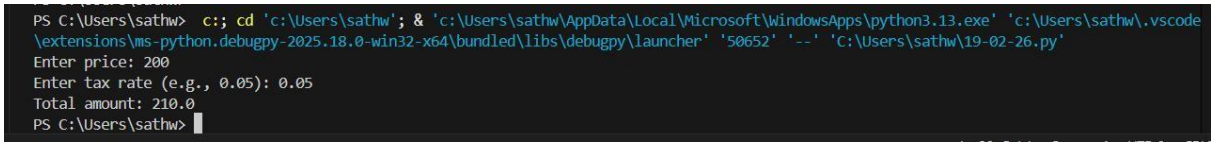
“Review the following Python script. Identify syntax errors, indentation issues, naming problems, and structural mistakes. Correct the code and explain each fix according to PEP 8 standards.”

Corrected Code (With Input)



```
C: > Users > sathw > 19-02-26.py > ...
#“Review the following Python script. Identify syntax errors, indentation issues, naming problems, and structural mistakes. Co
def calculate_total(price: float, tax: float) -> float:
    """
    Calculate total price including tax.
    """
    total = price + (price * tax)
    return total


if __name__ == "__main__":
    try:
        price_input = float(input("Enter price: "))
        tax_input = float(input("Enter tax rate (e.g., 0.05): "))

        result = calculate_total(price_input, tax_input)
        print("Total amount:", result)

    except ValueError:
        print("Invalid input! Please enter numeric values.")

```

OUTPUT:



```
PS C:\Users\sathw>  c:; cd 'c:\Users\sathw'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode
\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50652' '--' 'C:\Users\sathw\19-02-26.py'
Enter price: 200
Enter tax rate (e.g., 0.05): 0.05
Total amount: 210.0
PS C:\Users\sathw>
```

Task 2: Performance-Oriented Code Review

Scenario

A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description

You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

• Analyze the logic for performance bottlenecks

• Refactor the code for better time complexity

• Preserve the correctness of the output

Ask the AI to explain:

• Why the original approach was inefficient

• How the optimized version improves performance

Expected Outcome

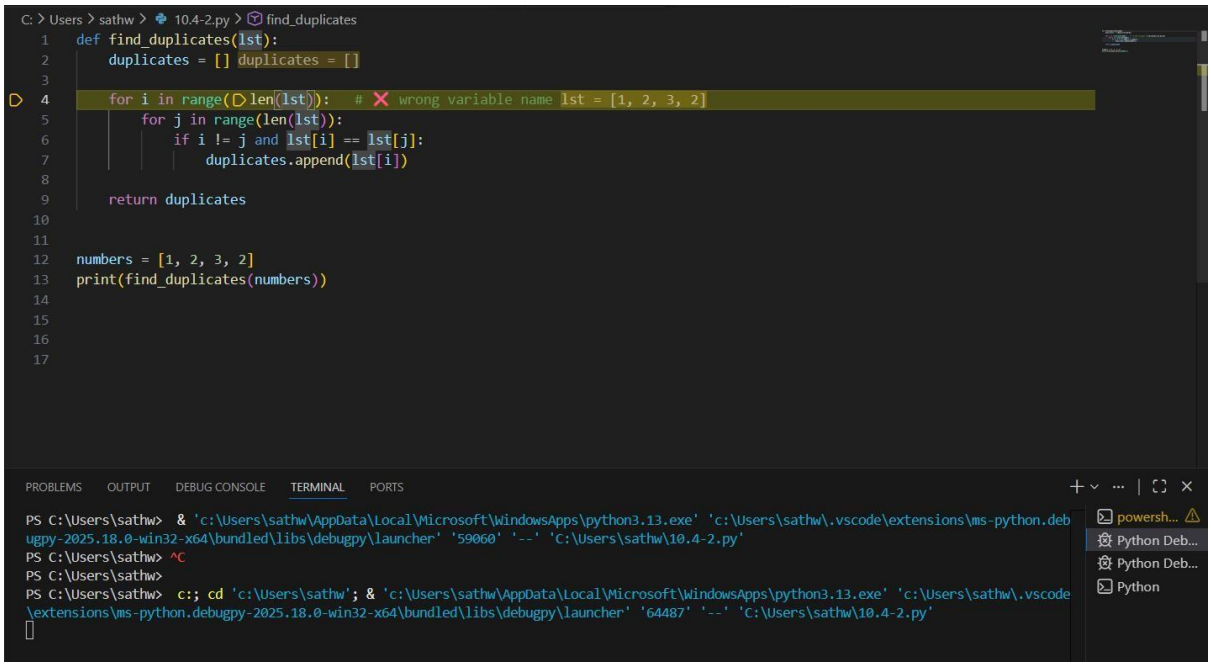
• Optimized duplicate-detection logic (e.g., using sets or hash-based structures)

• Improved time complexity

• AI explanation of performance improvement

• Clean, readable implementation

Original Inefficient Code



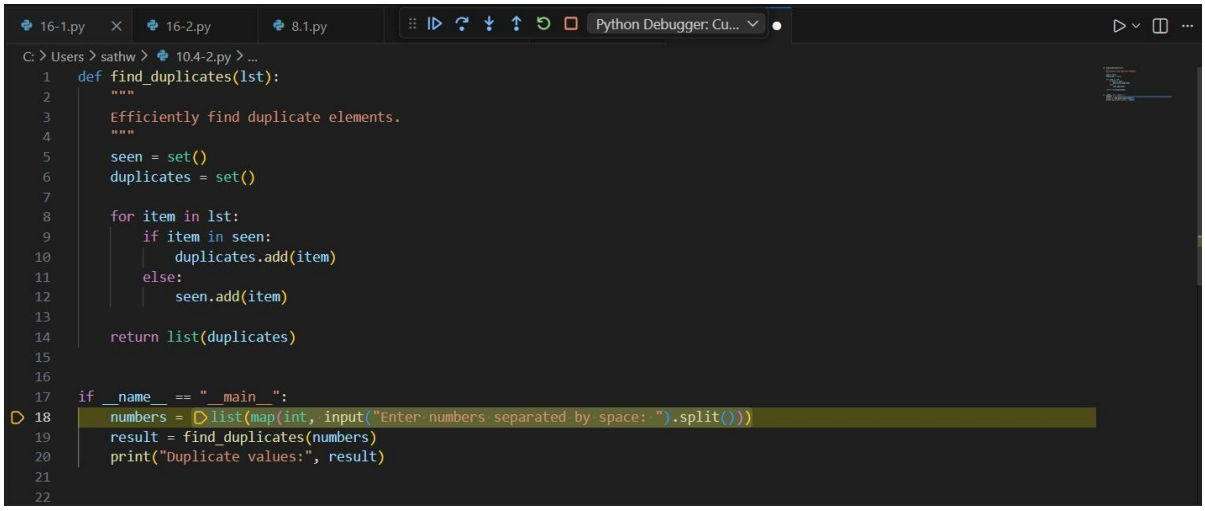
AI Identified Issues

- Nested loops → $O(n^2)$
- Duplicate values repeated
- Inefficient for large lists

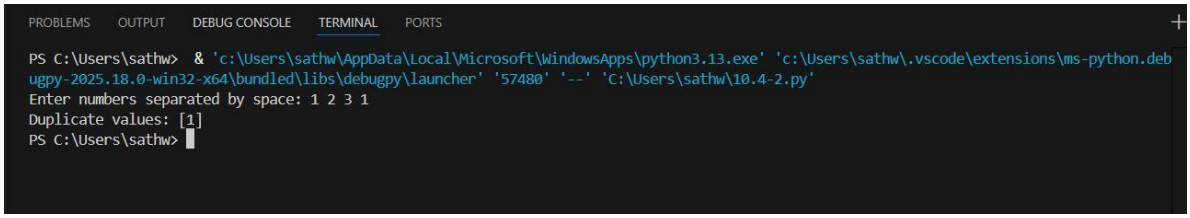
Prompt Used

“Analyze the performance of this duplicate detection function. Identify bottlenecks and refactor it to improve time complexity while preserving correctness.”

Optimized Code (With Input)



OUTPUT:



Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

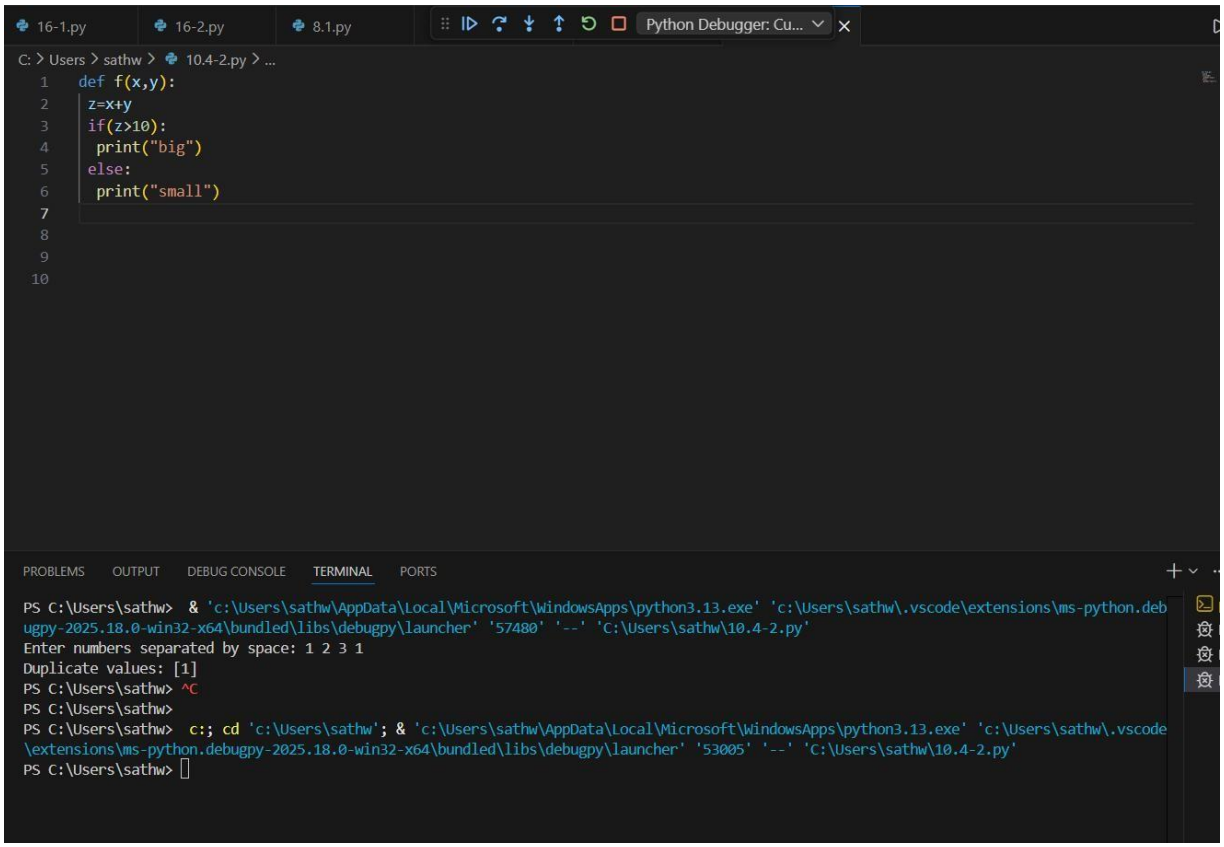
Use AI-assisted review to:

- Refactor the code for clarity
- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code
- Descriptive function and variable names
- Proper indentation and formatting
- Docstrings explaining the function purpose
- AI explanation of readability improvements

Original Inefficient Code



AI Identified Issues

- Unclear function name
- Poor indentation
- No documentation
- Cryptic variables

Prompt Used

“Refactor this Python function to improve readability, apply PEP 8 standards, use meaningful variable names, and add proper documentation.”

Optimized Code (With Input)

```python
#Refactor this Python function to improve readability, apply PEP 8 standards, use meaningful variable names, and add proper do
def check_sum_size(number1: int, number2: int) -> None:
    """
    Check whether the sum of two numbers is greater than 10.
    """
    total_sum = number1 + number2

    if total_sum > 10:
        print("The sum is large.")
    else:
        print("The sum is small.")


if __name__ == "__main__":
    try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))
        check_sum_size(num1, num2)

    except ValueError:
        print("Please enter valid integers.")
```

OUTPUT:

```
Enter first number: 3
Enter second number: 5
Enter first number: 3
Enter second number: 5
Enter second number: 5
The sum is small.
PS C:\Users\sathw> ^C
PS C:\Users\sathw>
PS C:\Users\sathw>  c:; cd 'c:\Users\sathw'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode
\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58863' '--' 'C:\Users\sathw\10.4-2.py'
Enter first number: 9
Enter second number: 5
The sum is large.
PS C:\Users\sathw>
```

Task 4: Secure Coding and Reliability Review

Scenario

A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description

You are given a Python script that:

- Uses unsafe SQL query construction
- Has no input validation
- Lacks exception handling

Use AI tools to:

• Identify security vulnerabilities

• Refactor the code using safe coding practices

• Add proper exception handling

• Improve robustness and reliability

Expected Outcome

• Secure SQL queries using parameterized statements

• Input validation logic

• Try-except blocks for runtime safety

• AI-generated explanation of security improvements

• Production-ready code structure

Original Inefficient Code

AI Identified Issues

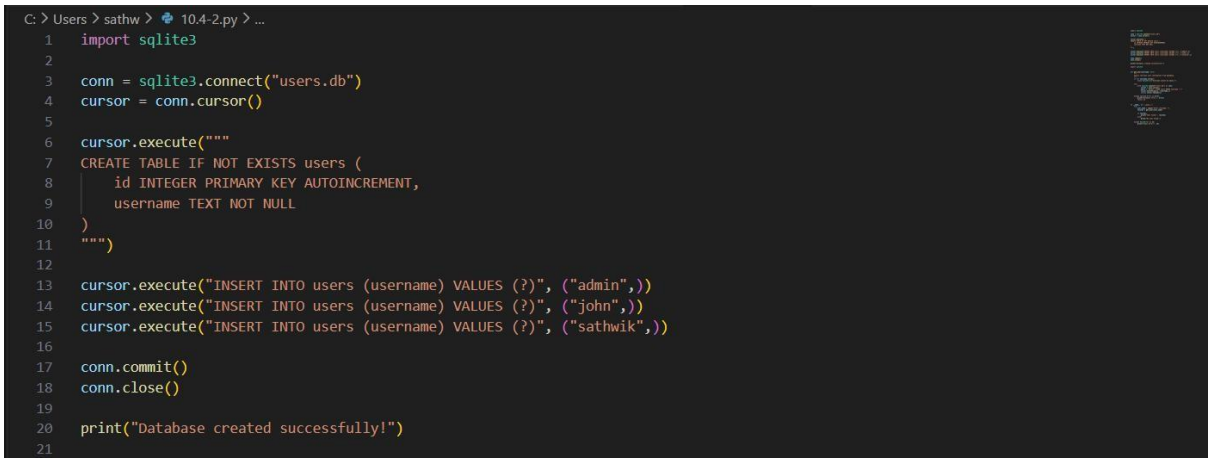
- SQL Injection vulnerability
- No validation
- No exception handling
- Connection not safely closed

Prompt Used

“Identify security vulnerabilities in this database code. Refactor using parameterized queries, input validation, and exception handling.”

CREATE A DATABASE FILE:

CODE WITH INPUT:



```
C: > Users > sathw > 10.4-2.py > ...
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL
)
""")

cursor.execute("INSERT INTO users (username) VALUES (?)", ("admin",))
cursor.execute("INSERT INTO users (username) VALUES (?)", ("john",))
cursor.execute("INSERT INTO users (username) VALUES (?)", ("sathwik",))

conn.commit()
conn.close()

print("Database created successfully!")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Database created successfully!
```

Optimized Code (With Input)

```python
import sqlite3

def get_user(username: str):
    """
    Safely retrieve user information from database.
    """
    if not username.strip():
        raise ValueError("Username cannot be empty.")

    try:
        with sqlite3.connect("users.db") as conn:
            cursor = conn.cursor()
            query = "SELECT * FROM users WHERE username = ?"
            cursor.execute(query, (username,))
            return cursor.fetchall()

    except sqlite3.Error as error:
        print("Database error:", error)
        return []

if __name__ == "__main__":
    try:
        user_name = input("Enter username: ")
        records = get_user(user_name)

        if records:
            print("User found:", records)
        else:
            print("No user found.")

    except ValueError as ve:
        print("Input Error:", ve)
```

OUTPUT:

```
Enter username: sathwik
User found: [(3, 'sathwik'), (6, 'sathwik')]
PS C:\Users\sathw>
```

Task 5: AI-Based Automated Code Review Report

Scenario

Your team uses AI tools to perform automated preliminary code reviews

before human review, to improve code quality and consistency across

projects.

Task Description

You are provided with a poorly written Python script.

Using AI-assisted review:

• Generate a structured code review report that evaluates:

o Code readability

o Naming conventions

o Formatting and style consistency

o Error handling

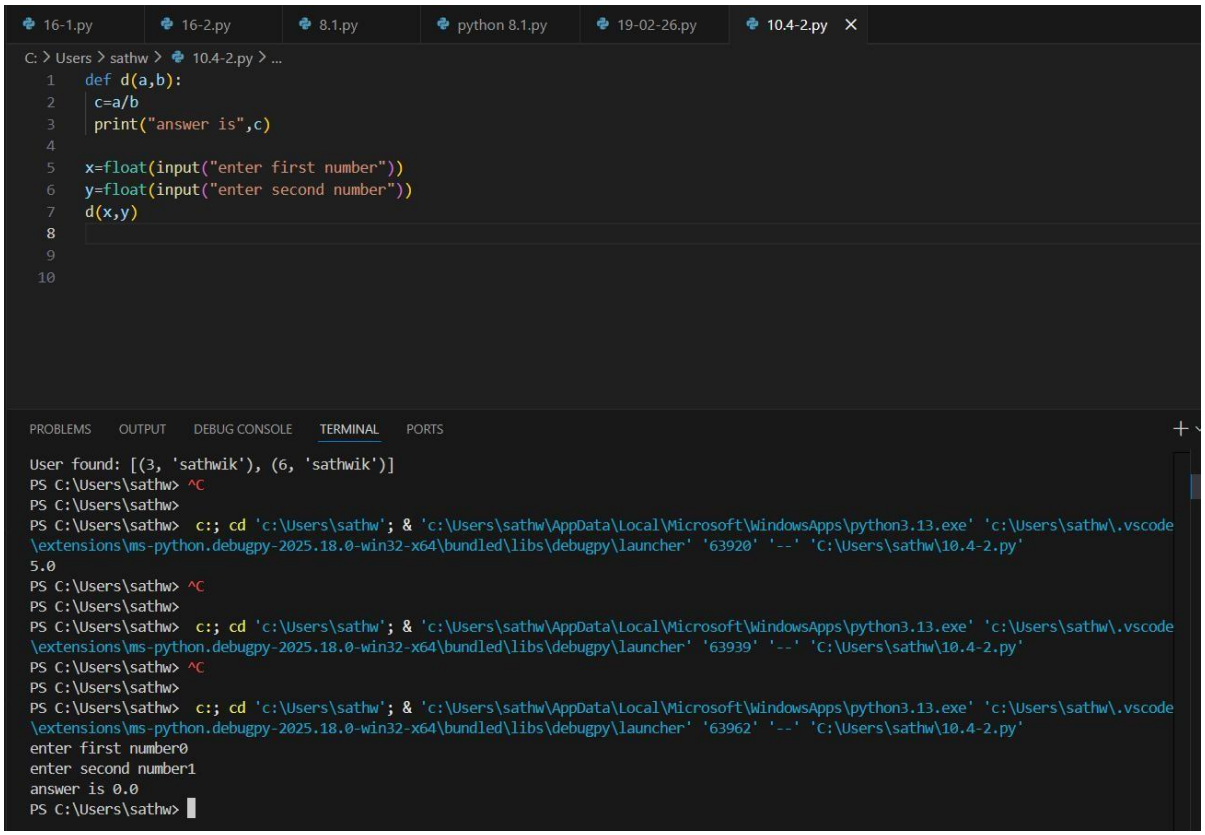
o Documentation quality

o Maintainability

The task is not just to fix the code, but to analyze and report on quality

Issues

Original Inefficient Code



AI Code Review Report

1. Readability Issues

- Function name not descriptive
- No documentation

2. Naming Problems

- Variables unclear

3. Formatting Issues

- Not PEP 8 compliant

4. Error Handling Risks

- Division by zero crash possible

5. Maintainability Concerns

- Not reusable
- No return value

Prompt Used for AI Review

"**Generate a structured code review report for the following Python script. Evaluate readability, naming conventions, formatting, style consistency, error handling, documentation quality, maintainability, and identify code smells and risk areas. Do not just fix the code — provide detailed analysis.**"

Optimized Code (With Input)

C: > Users > sathw > 10.4-2.py > ...

```python
def divide_numbers(dividend: float, divisor: float) -> float:
    """
    Divide two numbers safely.

    Args:
        dividend (float): Number to be divided.
        divisor (float): Number to divide by.

    Returns:
        float: Result of division.

    Raises:
        ValueError: If divisor is zero.
    """
    if divisor == 0:
        raise ValueError("Divisor cannot be zero.")

    return dividend / divisor


if __name__ == "__main__":
    try:
        num1 = float(input("Enter dividend: "))
        num2 = float(input("Enter divisor: "))

        result = divide_numbers(num1, num2)
        print("Result:", result)

    except ValueError as error:
        print("Error:", error)
```

OUTPUT:

```
PS C:\Users\sathw>  c:; cd 'c:\Users\sathw'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode
\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57235' '--' 'C:\Users\sathw\10.4-2.py'
Enter dividend: 50
Enter divisor: 5
Result: 10.0
PS C:\Users\sathw>
```